

FUNDAMENTALS OF C PROGRAMING

INTRODUCTION

- ▣ Developed at AT & T's Bell Laboratories of USA in 1972.
- ▣ Designed and written by Dennis Ritchie.
- ▣ C is compiled, structural, middle level programming language.

INTRODUCTION

There are so many languages like C++, Java etc

WHY SHOULD I STUDY C?

WHY SHOULD I STUDY C

- ▣ Learning C++ or Java directly is not an easy task.
- ▣ Learning the basic language elements makes your journey to Object Oriented languages a lot easier.

WHY SHOULD I STUDY C

- ▣ No one can beat C when it comes to PERFORMANCE.
- ▣ Major parts of OS like WINDOWS, LINUS and UNIX are written in C.
- ▣ If you have to interact with the Hardware(Microwave oven or washing machine coding) without compromising the performance, C would be your first choice.

PROGRAM

- ▣ A list of detailed instructions that tell the computer what to do.
- ▣ A Program is to a computer what a recipe is to a cook.
- ▣ Also known as **CODE**.
- ▣ **Advantage-** Do exactly what you want done

PROGRAMMING PROCESS

1. EDITOR

- ▣ Similar to Word Processor.
- ▣ Software that enables the typing of Programs.
- ▣ Easily type, change and save programs.

2. PREPROCESSOR

- ▣ The Program must be routed through a pre-processor before being compiled.
- ▣ Pre-processor reads a program's **preprocessor directives** to control the compilation

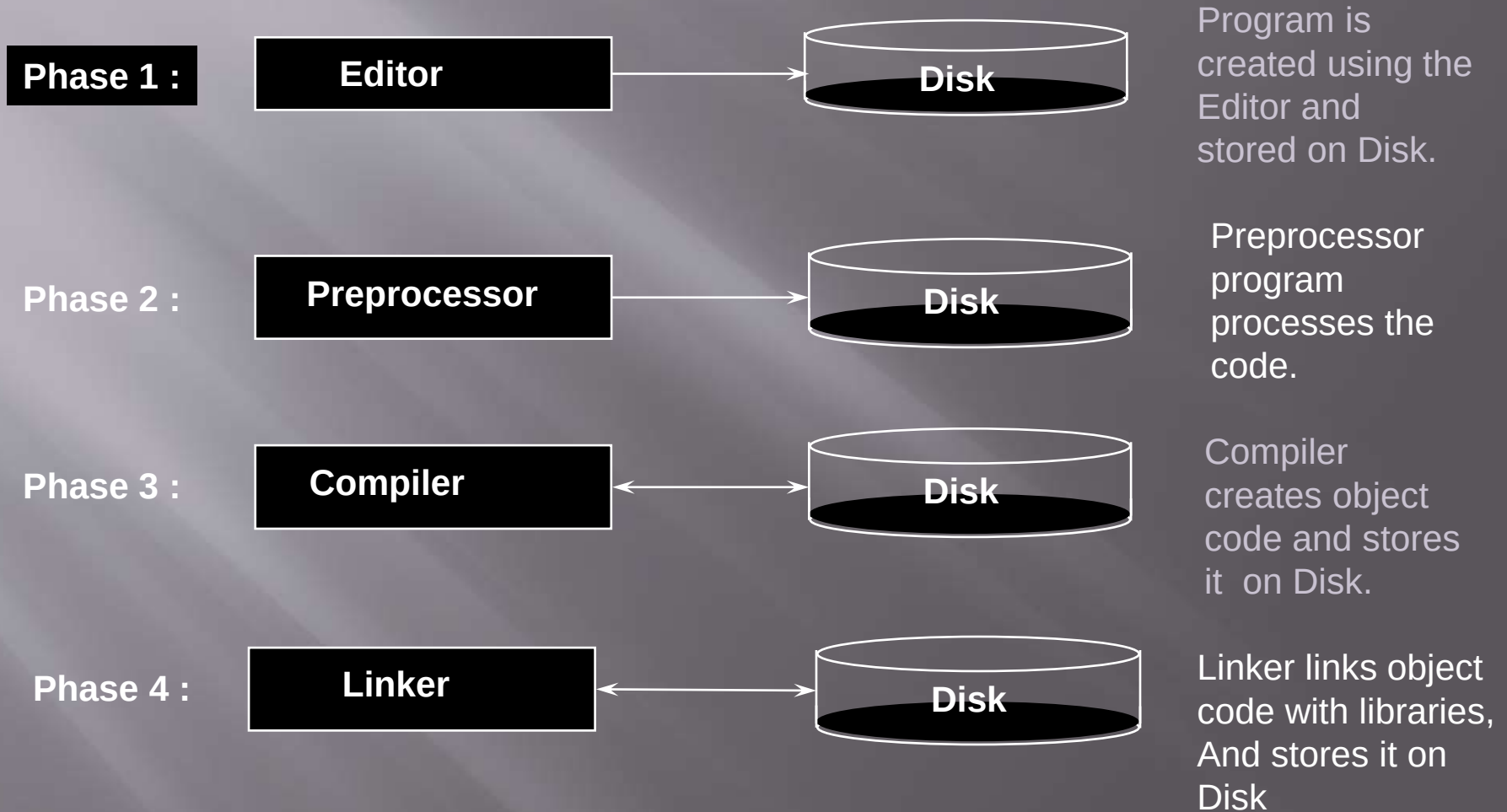
3. COMPILER

- ▣ Converts your Program to low-level machine instructions.
- ▣ **Object Code** is created and stored on the disk.

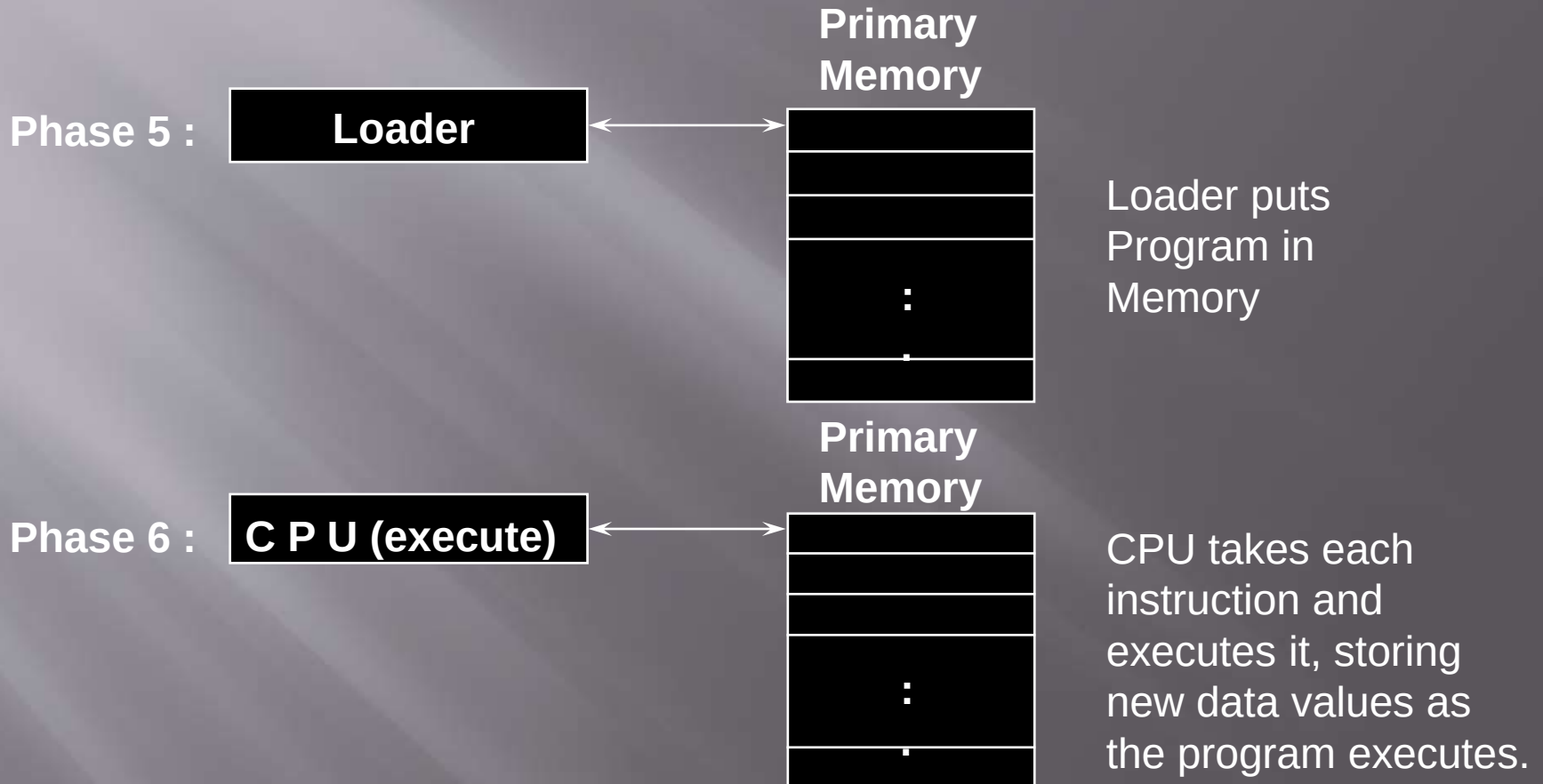
4. LINKING (FINAL STEP)

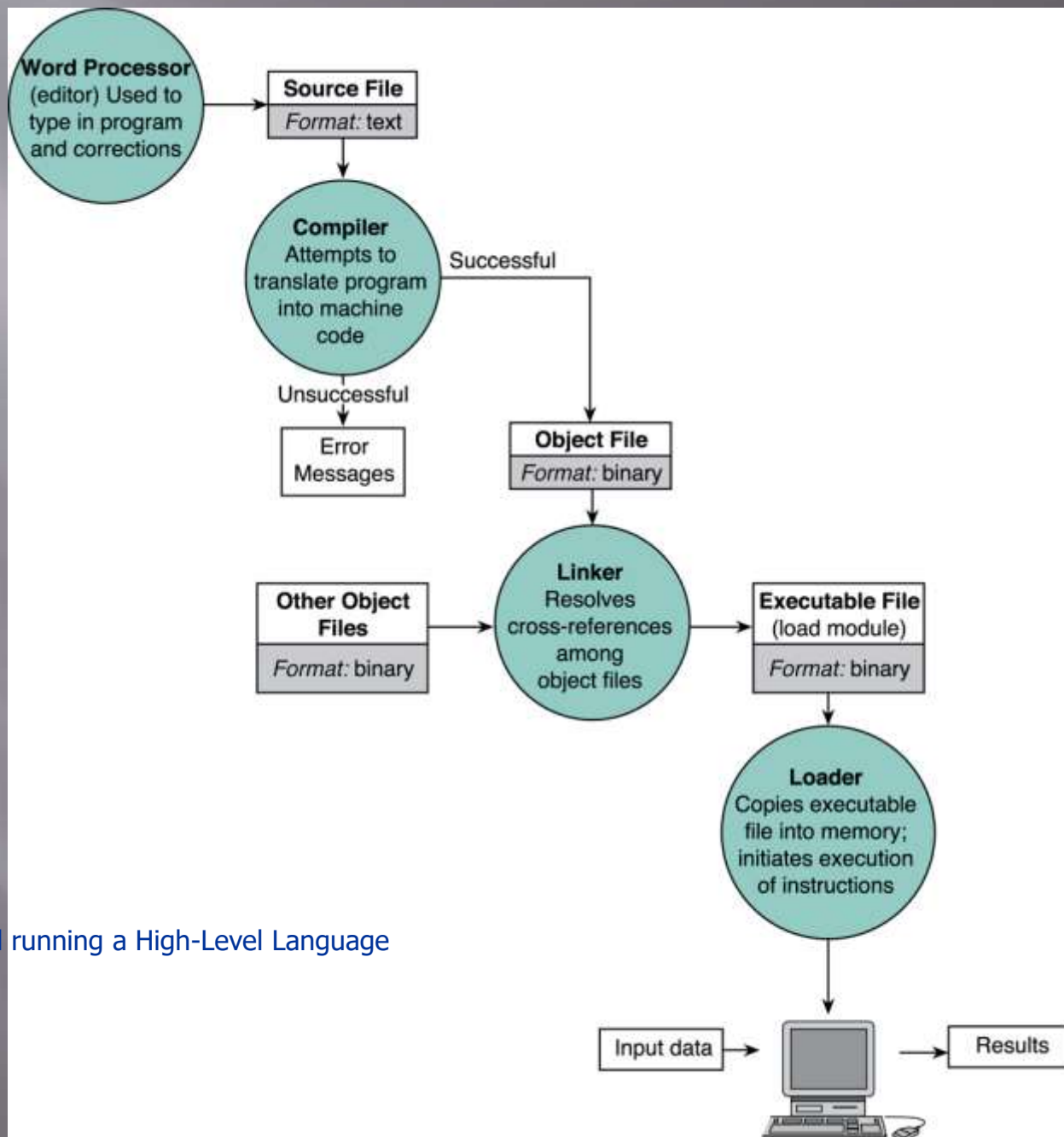
- ▣ After compiling and before running.
- ▣ Supplies runtime information to the compiled program.
- ▣ Linker links the Object code with libraries, and creates an executable code.

C Development Environment



C Development Environment cont





Entering, translating, and running a High-Level Language Program

AVOIDING AND REDUCING ERRORS

- ▣ When writing a Program, **Accuracy** is imperative!!
- ▣ Any mistake can cause Program Failure.
- ▣ The computer is very literal. Can do only what you tell it to do.

MOST COMMON ERROR

- ▣ Syntax – The rules governing the formation of statements in a programming language.
- ▣ Usually a typing error
- ▣ C informs you of the error with an error message. The programmer must know what to do to fix the mistake.

DEFINITION

- ▣ Bug- A Program mistake
- ▣ Debug- Correcting Program mistakes.

GETTING STARTED

C program skeleton

- ▣ In short, the basic skeleton of a C program looks like this:

```
#include <stdio.h>
```

← Preprocessor directives

```
void main(void)
```

← Function main

```
{
```

← Start of segment

```
    statement(s);
```

```
}
```

← End of segment

Preprocessor directives

- ▣ a C program line begins with # provides an instruction to the C preprocessor
- ▣ It is executed **before** the actual compilation is done and has some functions defined in them.
- ▣ Two most common directives :
 - #include
 - #define
- ▣ In our example (#include<stdio.h>) identifies the *header* file for standard input and output needed by the printf().
- ▣ Header file is given an extension .h
- ▣ C Source file is given an extension .c

Main function

- ▣ This is the entry point of a program
- ▣ When a file is executed, the start point is the main function
- ▣ From main function the flow goes as per the programmers choice.
- ▣ Main function is compulsory for any c program

The curly braces { }

- ▣ Identify a *segment/body* of a program
 - The start and end of a function
 - The start and end of the selection or repetition block.
- ▣ Since the opening brace indicates the **start** of a segment with the closing brace indicating the **end** of a segment, **there must be just as many opening braces as closing braces** (this is a common mistake of beginners)

Statement

- ▣ A specification of an action to be taken by the computer as the program executes.
- ▣ Each statement in C needs to be terminated with semicolon (;)
- ▣ Example:

```
#include <stdio.h>
```

```
void main(void)
```

```
{
```

```
    printf("I love programming\n");
```

```
    printf("You will love it too once ");
```

```
    printf("you know the trick\n");
```

```
}
```

statement

statement

statement

Statement cont...

- ▣ Statement has two parts :
 - Declaration
 - ▣ The part of the program that tells the compiler the names of memory cells in a program
 - Executable statements
 - ▣ Program lines that are converted to machine language instructions and executed by the computer

Identifiers

- ▣ Words used to represent certain program entities (variables, function names, etc).
- ▣ Example:
 - `int my_name;`
 - ▣ `my_name` is an identifier used as a program variable
 - `void CalculateTotal(int value)`
 - ▣ `CalculateTotal` is an identifier used as a function name

Rules for naming identifiers

Rules	Example
Can contain a mix of characters and numbers. However it cannot start with a number	H2o
First character must be a letter or underscore	Number1; _area
Can be of mixed cases including underscore character	XsquAre my_num
Cannot contain any arithmetic operators	R*S+T
... or any other punctuation marks...	#@x%!!
Cannot be a C keyword/reserved word	struct; printf;
Cannot contain a space	My height
... identifiers are case sensitive	Tax != tax

Basic Data Types

- ▣ There are 4 basic *data types* :
 - int
 - float
 - double
 - char
- ▣ **int**
 - used to declare numeric program variables of integer type
 - whole numbers, positive and negative
 - keyword: int
int number;
number = 12;

Basic Data Types cont...

▣ **float**

- fractional parts, positive and negative
- keyword: float
float height;
height = 1.72;

▣ **double**

- used to declare floating point variable of higher precision or higher range of numbers
- exponential numbers, positive and negative
- keyword: double
double valuebig;
valuebig = 12E-3;

Basic Data Types cont...

▣ char

- equivalent to 'letters' in English language
- Example of characters:
 - ▣ Numeric digits: 0 - 9
 - ▣ Lowercase/uppercase letters: a - z and A - Z
 - ▣ Space (blank)
 - ▣ Special characters: , . ; ? " / () [] { } * & % ^ < > etc
- single character
- keyword: char
char my_letter;
my_letter = 'U';

— The declared character must be enclosed within a single quote!

- ▣ In addition, there are **void**, **short**, **long**, etc.

CONSTANTS, VARIABLES & KEYWORDS

CONSTANTS

- ▣ Alphabets, numbers and special symbols are combined to form constants, variables and keywords.
- ▣ A constant is an entity that doesn't change.
- ▣ Example:-
 - ✓ Integer constants – 28, 108, 357 etc.
 - ✓ Real constants – 25.004, 100.258 etc.
 - ✓ Character constants – 'a', 'A', 'z', '=', '9'.

VARIABLES

- ▣ A Variable is an entity that does change.
- ▣ In programming we have to store the results of calculations in the computer memory.
- ▣ Variables are names given to memory locations

Memory cells

X 684521

10		

Variables

- ▣ **Variable** → a name associated with a memory cell whose value can change
- ▣ **Variable Declaration:** specifies the type of a variable
 - Example: `int num;`
- ▣ **Variable Definition:** *assigning* a value to the declared *variable*
 - Example: `num = 5;`

TYPES OF VARIABLES

- ▣ The memory cells can contain Integer, Floating Point and Character constants.
- ▣ Type of variable is nothing but the type of constant that the variable can store.

KEYWORDS

- ▣ By using Keywords, we can construct meaningful statements.
- ▣ These are the words whose meaning has already been explained to the c compiler.
eg- break, for, if, int, char, float etc.
- ▣ Keywords cannot be used as Variable names.

TYPES OF VARIABLES

VARIABLE TYPE	KEYWORD	EXAMPLE
Integer	int	int a; int age;
Real or Floating Point	float	float x; float average;
Character	char	char s; char initial;

#define

- ▣ You may also associate constant using #define preprocessor directive

```
#include <stdio.h>
#define pi 3.412
```

```
void main(void)
{
    double height, radius, base, volume;

    printf("Enter the height and radius of the cone:");
    scanf("%lf %lf",&height,&radius);

    base = pi * radius * radius;
    volume = (1.0/3.0) * base * height;

    printf("\nThe volume of a cone is %f ", volume);
}
```

Input/Output Functions

- ▣ A C function that performs an input or output operation
- ▣ A few functions that are pre-defined in the header file `stdio.h` such as :
 - `printf()`
 - `scanf()`
 - `getchar()` & `putchar()`

The printf function

- ▣ Used to send data to the standard output (usually the monitor) to be printed according to specific format.
- ▣ General format:
 - `printf("string literal");`
 - ▣ A sequence of any number of characters surrounded by double quotation marks.
 - `printf("format string", variables);`
 - ▣ Format string is a combination of text, conversion specifier and escape sequence.

The printf function cont...

▣ Example:

- `printf("Thank you");`
- `printf ("Total sum is: %d\n", sum);`
 - ▣ `%d` is a placeholder (conversion specifier)
 - marks the display position for a type integer variable
 - ▣ `\n` is an escape sequence
 - moves the cursor to the new line

Escape Sequence

Escape Sequence	Effect
\a	Beep sound
\b	Backspace
\f	Formfeed (for printing)
\n	New line
\r	Carriage return
\t	Tab
\v	Vertical tab
\\	Backslash
\"	“ sign
\o	Octal decimal
\x	Hexadecimal
\O	NULL

Placeholder / Conversion Specifier

No	Conversion Specifier	Output Type	Output Example
1	%d	Signed decimal integer	76
2	%i	Signed decimal integer	76
3	%o	Unsigned octal integer	134
4	%u	Unsigned decimal integer	76
5	%x	Unsigned hexadecimal (small letter)	9c
6	%X	Unsigned hexadecimal (capital letter)	9C
7	%f	Integer including decimal point	76.0000
8	%e	Signed floating point (using e notation)	7.6000e+01
9	%E	Signed floating point (using E notation)	7.6000E+01
10	%g	The shorter between %f and %e	76
11	%G	The shorter between %f and %E	76
12	%c	Character	'7'
13	%s	String	'76'

The scanf function

- ▣ Read data from the standard input device (usually keyboard) and store it in a variable.
- ▣ General format:
 - `scanf("Format string", &variable);`
- ▣ Notice ampersand (&) operator :
 - C address of operator
 - it passes the address of the variable instead of the variable itself
 - tells the `scanf()` where to find the variable to store the new value

The scanf function cont...

- Example :

```
int age;  
printf("Enter your age: ");  
scanf("%d", &age);
```

- Common Conversion Identifier used in printf and scanf functions.

	printf	scanf
int	%d	%d
float	%f	%f
double	%f	%lf
char	%c	%c
string	%s	%s

The scanf function cont...

- ▣ If you want the user to enter more than one value, you serialise the inputs.

- ▣ Example:

```
float height, weight;
```

```
printf("Please enter your height and weight:");
```

```
scanf(" %f%f", &height, &weight);
```